Obtaining a Toolchain

Building a toolchain manually

- Building a cross-compiling toolchain manually is a fairly difficult process
- Lots of details to learn: many components to build with complicated configuration
- Typical process is:
  - Build dependencies of binutils/gcc (GMP, MPFR, ISL, etc.)
  - Build *binutils*
  - Build a baremetal, first stage *GCC*
  - Extract kernel headers from the Linux source code
  - Build the C library using the first stage GCC
  - Build the second stage and final *GCC* supporting the Linux OS and the C library.
- Many decisions to make about the components: C library, gcc and binutils versions, ABI, floating point mechanisms, etc. Not trivial to find correct combinations of these possibilities
- See the `Crosstool-NG documentation` for details on how toolchains are built.
- Talk: *Anatomy of Cross-Compilation Toolchains*, by Thomas Petazzoni, ELCE 2017, `video` and `slides`

Get a pre-compiled toolchain

- Solution that many people choose
  - Advantage: it is the simplest and most convenient solution
  - Drawback: you can't fine tune the toolchain to your needs
- Make sure the toolchain you find meets your requirements: CPU, endianness, C library, component versions, version of the kernel headers, ABI, soft float or hard float, etc.
- Some possibilities:
  - Toolchains packaged by your distribution, for example Ubuntu package `gcc-arm-linux-gnueabihf` or Fedora `gcc-arm-linux-gnu`. Often limited to ARM/ARM64 with glibc.
  - Bootlin's GNU toolchains, most CPU architectures, with glibc/uClibc/musl, `https://toolchains.bootlin.com`
  - `ARM and ARM64 toolchains released by ARM`

Example of toolchains from ARM: downloading

x86_64 Linux hosted cross compilers

AArch32 bare-metal target (arm-none-eabi)

- gcc-arm-10.3-2021.07-x86_64-arm-none-eabi.tar.xz
- gcc-arm-10.3-2021.07-x86_64-arm-none-eabi.tar.xz.asc

AArch32 target with hard float (arm-none-linux-gnueabihf)

- gcc-arm-10.3-2021.07-x86_64-arm-none-linux-gnueabihf.tar.xz
- gcc-arm-10.3-2021.07-x86_64-arm-none-linux-gnueabihf.tar.xz.asc

AArch64 ELF bare-metal target (aarch64-none-elf)

- gcc-arm-10.3-2021.07-x86_64-aarch64-none-elf.tar.xz
- gcc-arm-10.3-2021.07-x86_64-aarch64-none-elf.tar.xz.asc

AArch64 GNU/Linux target (aarch64-none-linux-gnu)

- gcc-arm-10.3-2021.07-x86_64-aarch64-none-linux-gnu.tar.xz
- gcc-arm-10.3-2021.07-x86_64-aarch64-none-linux-gnu.tar.xz.asc

AArch64 GNU/Linux target (aarch64_be-none-linux-gnu)

- gcc-arm-10.3-2021.07-x86_64-aarch64_be-none-linux-gnu.tar.xz
- gcc-arm-10.3-2021.07-x86_64-aarch64_be-none-linux-gnu.tar.xz.asc

From `Arm GNU Toolchains`

Example of toolchains from ARM: using

```
\$\_wget https://developer.arm.com/-/media/Files/downloads/gnu-a/10.3-2021.07/binrel/[...]
    [...]gcc-arm-10.3-2021.07-x86_64-arm-none-linux-gnueabihf.tar.xz

\$\_tar xf gcc-arm-10.3-2021.07-x86_64-arm-none-linux-gnueabihf.tar.xz

\$\_cd gcc-arm-10.3-2021.07-x86_64-arm-none-linux-gnueabihf/

\$\_./bin/arm-none-linux-gnueabihf-gcc -o test test.c

\$\_file test test: ELF 32-bit LSB executable, ARM, EABI5 version 1 (SYSV), dynamically linked, interpreter /
lib/ld-linux-armhf.so.3, [...]
   for GNU/Linux 3.2.0, with debug_info, not stripped
```

Toolchain building utilities Another solution is to use utilities

that **automate the process of building the toolchain**

- Same advantage as the pre-compiled toolchains: you don't need to mess up with all the details of the build process
- But also offers more flexibility in terms of toolchain configuration, component version selection, etc.
- Allows to rebuild the toolchain if needed to fix a bug or security issue.
- They also usually contain several patches that fix known issues with the different components on some architectures
- Multiple tools with identical principle: shell scripts or Makefile that automatically fetch, extract, configure, compile and install the different components

Toolchain building utilities (2)

Crosstool-ng

- Rewrite of the older Crosstool, with a menuconfig-like configuration system
- Feature-full: supports uClibc, glibc and musl, hard and soft float, many architectures
- Actively maintained
- https://crosstool-ng.github.io/

Toolchain building utilities (2)

Crosstool-NG is a versatile (cross) toolchain generator. It supports many architectures and components and has a simple yet powerful menuconfig-style interface. Please read the introduction and refer to the documentation for more information.

See what the users of crosstool-NG have to say!

Latest sources, bugs, questions? Head over to Crosstool-NG at GitHub!

News

May 7, 2022

Released 1.25.0

Get the 1.25.0 release as bz2 (PGP signature) (md5, sha1, sha512) or xz (PGP signature) (md5, sha1, sha512) .

continued ...

Apr 22, 2022

Released 1.25.0_rc2

Get the 1.25.0_rc2 release as bz2 (PGP signature) (md5, sha1, sha512) or xz (PGP signature) (md5, sha1, sha512) .

Mar 24, 2022

Released 1.25.0_rc1

Get the 1.25.0_rc1 release as bz2 (PGP signature) (md5, sha1, sha512) or xz (PGP signature) (md5, sha1, sha512) .

Toolchain building utilities (3) Many root filesystem build

systems also allow the construction of a cross-compiling toolchain

- **Buildroot**
  - Makefile-based. Can build glibc, uClibc and musl based toolchains, for a wide range of architectures. Use `make sdk` to only generate a toolchain.
  - `https://buildroot.org`
- **PTXdist**
  - Makefile-based, maintained mainly by *Pengutronix*, supporting only glibc and uClibc (version 2023.01 status)
  - `https://www.ptxdist.org/`
- **OpenEmbedded / Yocto Project**
  - A featureful, but more complicated build system, supporting only glibc and musl.
  - `https://www.openembedded.org/`
  - `https://www.yoctoproject.org/`

Crosstool-NG: download

- Getting Crosstool-NG

```
\$\_git clone https://github.com/crosstool-ng/crosstool-ng.git
```

- Using a well-known stable version

```
\$\_cd crosstool-ng
\$\_git checkout crosstool-ng-1.27.0
```

- As we're fetching from Git, the `configure` script needs to be generated:

```
\$\_./bootstrap
```

Crosstool-NG: installation

- Installation can be done:
  - system-wide, for example in `/usr/local`, the `ct-ng` command is then available globally

```
\$\_./configure
\$\_make
\$\_sudo make install
```

  - or just locally in the source directory, the `ct-ng` command will be invoked from this directory

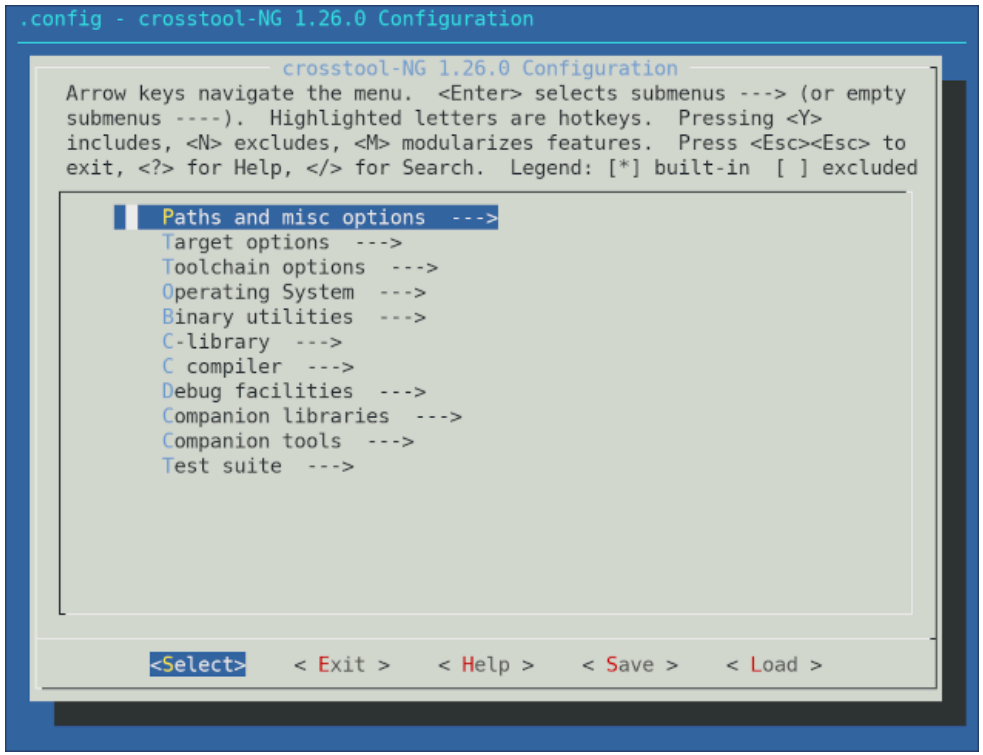
```
\$\_./configure --enable-local
\$\_make
```

- In our labs, we will use the second method
- Note: the `make` invocation doesn't build any toolchain, it builds the `ct-ng` executable.

Crosstool-NG: toolchain configuration

- Once installed, the `ct-ng` tool allows to configure and build an arbitrary number of toolchains
- Its configuration system is based on *kconfig*, like the Linux kernel configuration system
- Configuration of the toolchain to build stored in a `.config` file
- Example configurations provided with Crosstool-NG
  - List: `./ct-ng list-samples`
  - Load an example: `./ct-ng <sample-name>`, replaces `.config`
  - For example `./ct-ng aarch64-unknown-linux-gnu`
  - No sample loaded $arrow.r$_default Crosstool-NG configuration is a bare-metal toolchain for the *Alpha* CPU architecture!
- The configuration can then be refined using either:
  - `./ct-ng menuconfig`
  - `./ct-ng nconfig`

Crosstool-NG: toolchain configuration




```
./ct-ng menuconfig
```

Crosstool-NG: toolchain building

Important toolchain contents

- `bin/`: cross compilation tool binaries
  - This directory can be added to your `PATH` to ease usage of the toolchain
  - Sometimes with symlinks for shorter names

```
arm-linux-gcc -> arm-cortexa7-linux-uclibcgnueabihf-gcc
```

- `<arch-tuple>/sysroot`: *sysroot* directory
  - `<arch-tuple>/sysroot/lib`: C library, GCC runtime, C++ standard library compiled for the target
  - `<arch-tuple>/sysroot/usr/include`: C library headers and kernel headers